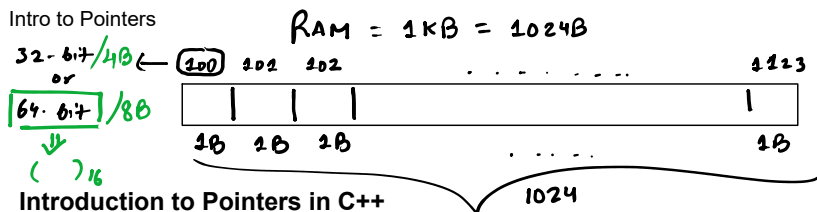
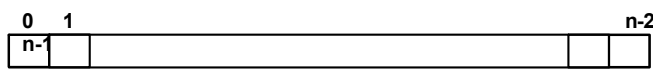


Intro to Pointers



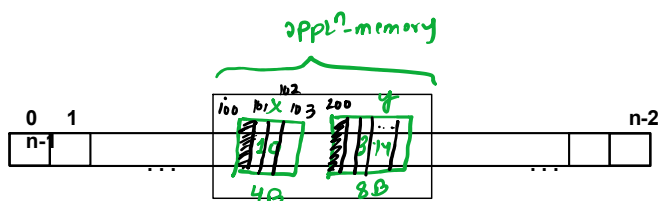
Introduction to Pointers in C++

- We can visualize the system memory (RAM) as a sequence of memory cells such that each memory cell is of **1B (8 bits)** and is addressable.



- These addresses depending on the system (and the compiler) are either of **32-bits** i.e. **4B** or **64-bits** i.e. **8B**. Moreover, computer use the hexa-decimal i.e. the base-16 number system to represent addresses.

- When we run a C++ program, a portion of system memory is allocated for program execution which is known as **application memory**.



⇒ 0x _____
⇒ 0b _____

main() {

int x = 10;

cout << x; // 100

double y = 3.14;

cout << y; // 200

}

- To know the address of a byte in C++, we use the **address of (&)** operator.

=

- When we print **address** of a **character** using the **cout <<**, C++ prints the contents of the memory stored at that address until it encounters the **null character ('0')**.

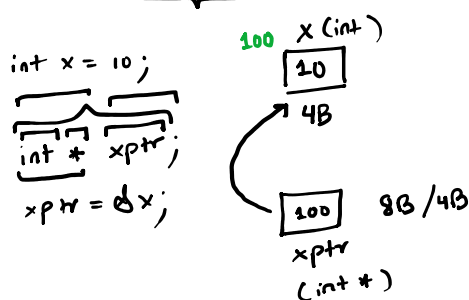
Pointers in C++

A **pointer** is a variable which is used to store **address** of another variable.

⇒

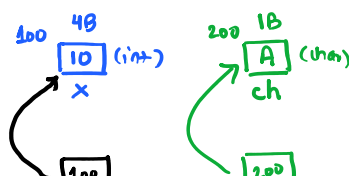
```
// syntax for pointer declaration
type name;
```

~~~~~



Since a pointer stores an address which are of **8B**, in C++, we can use a pointer of one type to store the address of a variable of another type via explicit type-casting.

```
int main() {
    int x = 10;
    char* ptr1 = (char*)&x;
    char ch = 'A';
```



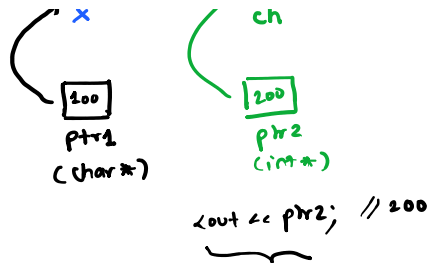
```

int x = 10;
char* ptr1 = (char*)&x;

char ch = 'A';
int* ptr2 = (int*)&ch;

return 0;

```



## Dereferencing a Pointer in C++

To dereference a pointer, i.e. to access the value of the variable whose address is stored by the pointer variable, we use the **dereference** operator (`*`).

```

int x = 10;

int * xptr = &x;

cout << x; // 10
cout << xptr; // 100
cout << *xptr; // 10

```

value at  $\frac{xptr}{100}$

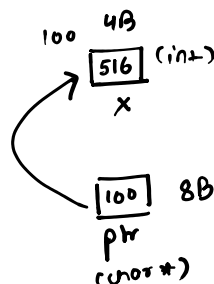
$$516 = 512 + 4$$

$\uparrow$   $2^9$   $\uparrow$   $2^2$

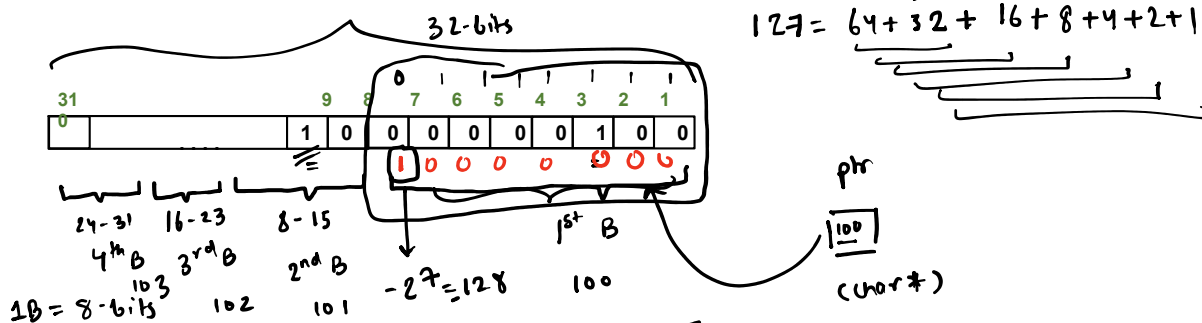
```

int main() {
    int x = 516;
    char* ptr = (char*)&x;
    cout << (int)*ptr << endl;
    return 0;
}

```



$$128 = 2^7$$



```

cout << (int)*ptr; // 4

```

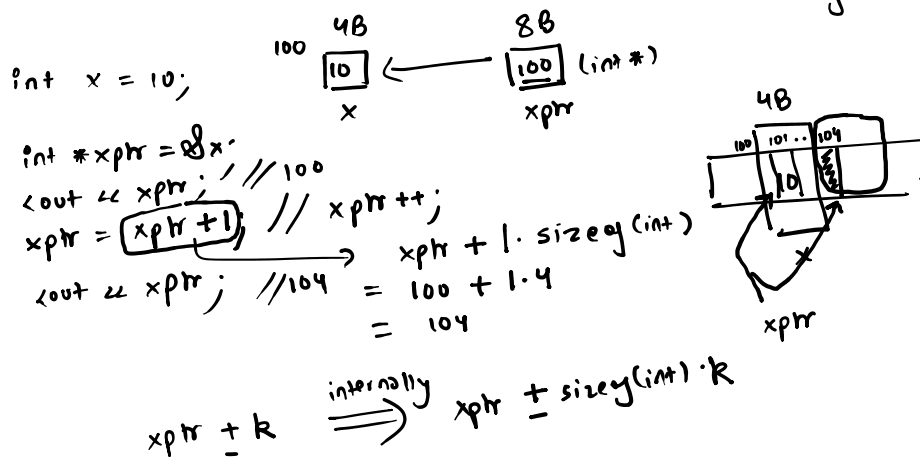
When we deref a char\*, what happens we go to addr stored in ptr and read only 1B of data since char in C++ is 1B

## Pointer Arithmetic

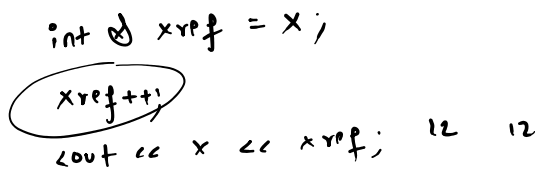
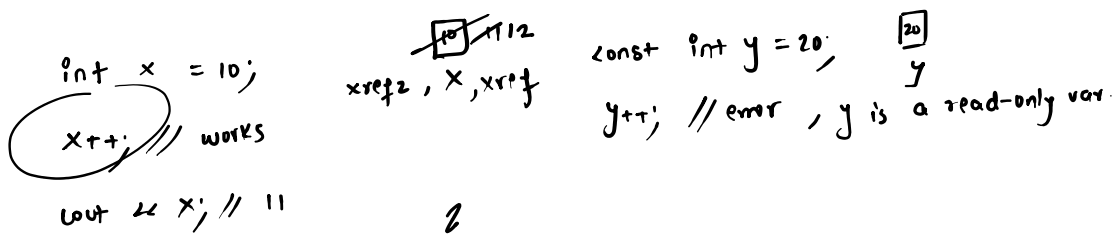
A pointer variable only supports arithmetic **addition** and **subtraction** operations. Moreover, in C++, we can also subtract two pointer variables.



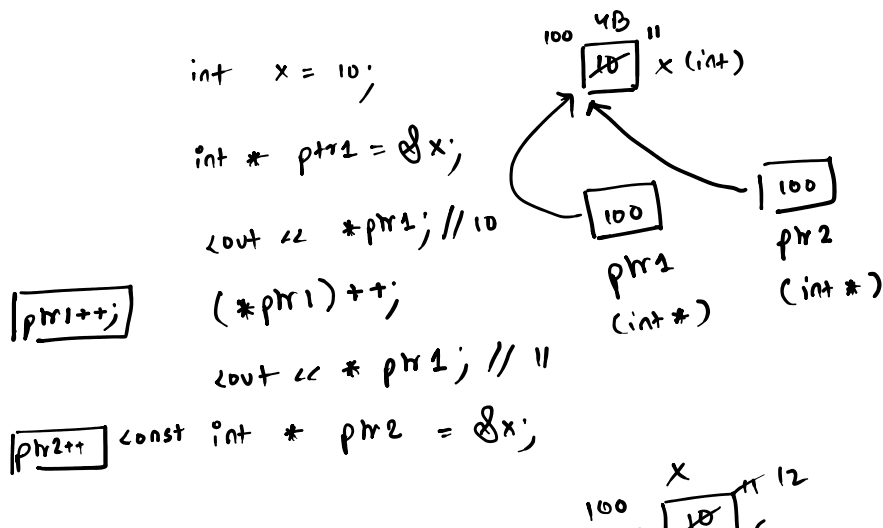
A pointer variable only supports arithmetic **addition** and **subtraction** operations. Moreover, in C++, we can also subtract two pointer variables.



`const Pointer` vs `Pointer to const` vs `const Pointer to const`



`const int &xref2 = x;`  
`xref2++;` // error

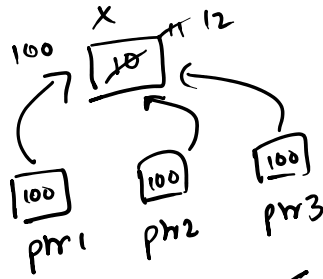


ptr2++ ~~const int \* ptr2 = &x;~~

```
int x = 10;

int *ptr1 = &x;
const int *ptr2 = &x;
int * const ptr3 = &x;

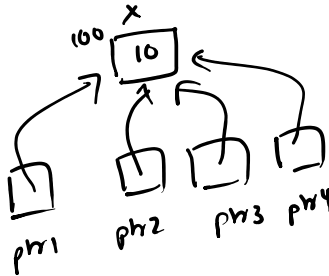
(*ptr1)++; ✓
*ptr2++; ✗
(*ptr3)++; ✓
```



```
ptr1++; ✓
ptr2++; ✓
ptr3++; ✗
```

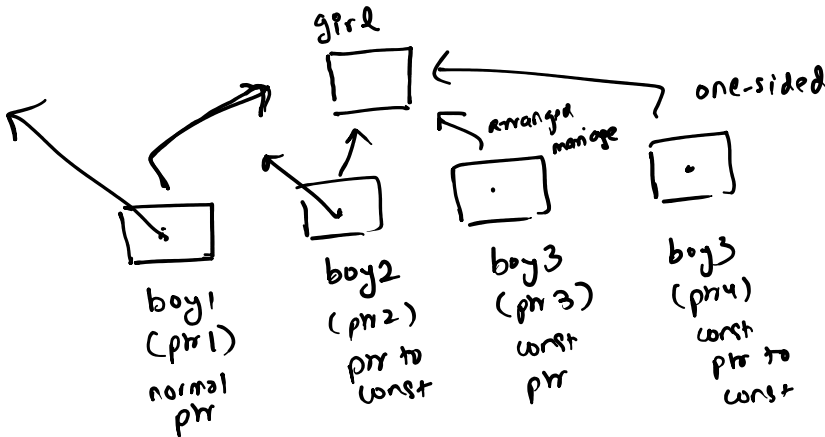
```
int x = 10;

int *ptr1 = &x;
const int *ptr2 = &x;
int * const ptr3 = &x;
const int * const ptr4 = &x;
```



```
(*ptr1)++; ✓
(*ptr2)++; ✗
(*ptr3)++; ✓
(*ptr4)++; ✗
```

```
ptr1++; ✓
ptr2++; ✓
ptr3++; ✗
ptr4++; ✗
```



## Pointers and Functions

pass by value

```
void increment(int a) {
    a++;
}

int main() {
    int a = 0;
```

pass by ref

```
void increment(int& ref) {
    ref++;
}

int main() {
    int a = 0;
```

```
int main() {
    int a = 0;
    => increment(a);
    cout << a << endl; //0
}
```

```
int main() {
    int a = 0;
    => increment(a);
    cout << a << endl; //1
}
```

passing addr by value

```
void increment(int* ptr) {
    (*ptr)++;
}

int main() {
    int a = 0;
    int* ptr = &a;
    increment(ptr);
    cout << a << endl; //1
}
```

passing addr by ref

```
void increment(int*& ptrRef) {
    (*ptrRef)++;
}

int main() {
    int a = 0;
    int* ptr = &a;
    increment(ptr);
    cout << a << endl; //1
}
```

return by value

```
int multiply(int a, int b) {
    int c = a * b;
    return c;
}

int main() {
    cout << multiply(2, 3) << endl; //6
    return 0;
}
```

```
int multiply(int a, int b) {
    int c = a * b; // local variable
    return c;
}

int main() {
    cout << multiply(2, 3) << endl;
    return 0;
}
```

```
int multiply(int a, int b) {
    int c = a * b; // local variable
    return &c;
}

int main() {
    cout << multiply(2, 3) << endl;
    return 0;
}
```

Dangling Ref. Problem

Dangling Pointer Problem

return addr by value

ret. addr by ref

```
int c; // global variable

int* multiply(int a, int b) {
    c = a * b;
    return c;
}

int main() {
    cout << multiply(2, 3) << endl;
    return 0;
}
```

```
int c; // global variable

int* multiply(int a, int b) {
    c = a * b;
    return &c;
}

int main() {
    cout << *multiply(2, 3) << endl;
    return 0;
}
```

```
int c; // global variable

int* multiply(int a, int b) {
    c = a * b;
    return &c;
}

int main() {
    cout << *multiply(2, 3) << endl;
    return 0;
}
```

## Void or Generic Pointers

A generic or void pointer (void\*) can be used to address of a variable of any type.

⇒

```
int main() {
    char ch = 'a';
    void* ptr = &ch;
    cout << ptr << endl; // 100
}
```



In C++, void pointers cannot be **dereferenced**.

✗✗

## Null Pointers

A **null pointer** is a pointer variable that contains the *null pointer value* and therefore it points to nowhere.

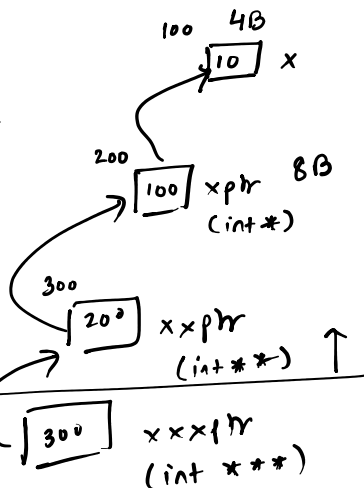
```
int main() {
    int* p = nullptr;
    int* q = 0;
    int* r = NULL; ✓
}
```

In C++, null pointers cannot be dereferenced.

## Pointer to Pointer

In C++, we can create a pointer to point to another pointer.

```
int main() {
    int x = 10;
    int* xptr = &x;
    int** xxptr = &xptr;
    int*** xxxptr = &xxptr;
    return 0;
}
```



*\*xptr;*  
*\*\*xxptr;*  
*\*\*\*xxxptr;*

## Pointers and Arrays

`int arr[] = { 10, 20, 30, 40, 50 }`

In C++, we can think of **name** of an array as a **pointer** to the element at the 0<sup>th</sup> index.

| index   | 0   | 1   | 2   | 3   | 4   |
|---------|-----|-----|-----|-----|-----|
| value   | 10  | 20  | 30  | 40  | 50  |
| address | 100 | 104 | 108 | 112 | 116 |

Diagram showing the array 'arr' with values and addresses. Arrows indicate the relationship between the array name and the first element, and between consecutive elements.

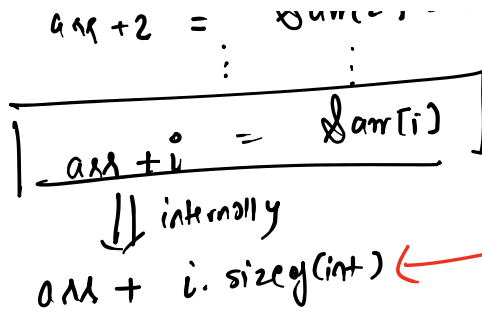
$$\begin{aligned} \text{arr} + 1 \\ \Downarrow \text{internally} \\ 100 + 1 \cdot 4 \\ = 104 \end{aligned}$$

$$\begin{aligned} \text{arr} &= \&\text{arr}[0] = 100 \\ \text{arr} + 1 &= \&\text{arr}[1] = 104 \\ \text{arr} + 2 &= \&\text{arr}[2] = 108 \\ &\vdots \\ &\vdots \end{aligned}$$

$$\begin{aligned} *arr &= arr[0] = 10 \\ *(arr + 1) &= arr[1] = 20 \\ *(arr + 2) &= arr[2] = 30 \end{aligned}$$

$$*(arr + i) = arr[i]$$

= 107



$$*(arr + i) = arr[i]$$

very fast operation / constant time

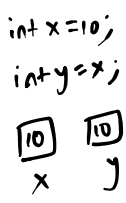
$$*ptr = 10 \quad *(ptr + 1) = 20$$

ptr[1] = 20

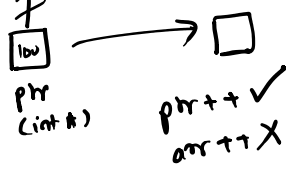
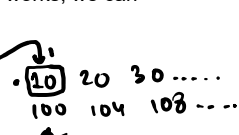
$$*(ptr + 2) = 30 \dots$$

ptr[2] = 30 \dots

Since the name of an array in C++ works exactly how a pointer works, we can assign an array name to a pointer.



```
int main() {  
    int arr[] = {10, 20, 30, 40, 50, 60, 70};  
    int n = sizeof(arr) / sizeof(int);  
    int* ptr = arr;  
    for(int i=0; i<n; i++) {  
        cout << ptr[i] << " "; // or *(ptr+i)  
    }  
    return 0;  
}
```



$$ptr[i] \Rightarrow *(ptr + i)$$

The **difference** between a pointer and array name is that while a pointer can be assigned a different address, the array name cannot be assigned anything.